Java 入门笔记

第十一课 · 类、对象、参数传递与方法重载

学习目标	前面的程序主要使用 `static` 方法处理数据。本课开始建立面向对象的基本模型：类描述一类事物，对象是根据类创建的具体实例，字段保存状态，方法描述行为。

目录

本课目标 2

6.1 从面向过程到面向对象 2

6.2 类的基本结构 2

6.3 创建和使用对象 3

6.4 构造方法 3

6.5 实例成员与 static 成员 4

6.6 Java 的参数传递：始终是值传递 4

6.7 方法重载 5

6.8 完整示例：学生类 6

常见错误 7

本课小结 7

自测题 7

整理日期：2026 年 11 月 1 日

本课目标

- 理解类、对象、字段、方法和构造方法的关系。
- 会定义简单类、创建对象并访问成员。
- 正确认识 Java 的值传递机制。
- 掌握方法重载的判定规则和常见歧义。

6.1 从面向过程到面向对象

面向过程关注“按什么步骤完成任务”，面向对象关注“由哪些对象协作完成任务”。两者并不互相排斥：对象的方法内部仍然由顺序、选择、循环等语句组成。

以学生信息为例：

```
学生类 Student
├── 状态（字段）：name、age、score
└── 行为（方法）：introduce、isPassed
```

类是设计说明，对象是运行时创建的具体实例。

6.2 类的基本结构

```
class Student {
    String name;
    int age;
    double score;

    void introduce() {
        System.out.println("我是 " + name + "，年龄 " + age);
    }

    boolean isPassed() {
        return score >= 60;
    }
}
```

成员	作用	示例
字段	保存对象状态	`String name;`
方法	描述对象行为	`void introduce()`
构造方法	创建对象时完成初始化	`Student(...)`

字段声明在类体中、方法之外。字段有默认值；方法中的局部变量必须先明确赋值再使用。

6.3 创建和使用对象

```
Student first = new Student();
first.name = "张三";
first.age = 20;
first.score = 86.5;

first.introduce();
System.out.println(first.isPassed());
```

`new Student()` 创建对象，`first` 保存对象引用。点运算符 `.` 用于访问对象的字段和方法。

```
引用变量 first ──────────> Student 对象
                           name = "张三"
                           age = 20
                           score = 86.5
```

不同对象拥有各自的实例字段：

```
Student first = new Student();
Student second = new Student();
first.name = "张三";
second.name = "李四";
```

修改 `first.name` 不会自动修改 `second.name`。

6.4 构造方法

构造方法用于创建对象时初始化字段，它与类同名，并且没有返回值类型。

```
class Student {
    String name;
    int age;
    double score;

    Student(String name, int age, double score) {
        this.name = name;
        this.age = age;
        this.score = score;
    }
}
```

```
Student student = new Student("张三", 20, 86.5);
```

`this` 表示当前对象。`this.name` 是字段，右侧 `name` 是构造方法参数。

如果类中没有显式声明任何构造方法，编译器会提供一个无参数默认构造方法。一旦自行声明构造方法，编译器就不再自动补充这个无参数构造方法。

6.5 实例成员与 static 成员

实例字段和实例方法属于具体对象，通常通过对象引用访问。`static` 成员属于类本身，通常通过类名访问。

```
class Counter {
    static int total;
    int value;

    void increase() {
        value++;
        total++;
    }
}
```

```
Counter first = new Counter();
Counter second = new Counter();
first.increase();
second.increase();

System.out.println(first.value);  // 1
System.out.println(second.value); // 1
System.out.println(Counter.total); // 2
```

`main` 是 `static` 方法，所以不能在没有对象的情况下直接访问实例字段或实例方法。

6.6 Java 的参数传递：始终是值传递

调用方法时，Java 会把实参的值复制给形参。对于基本类型，复制的是数值；对于引用类型，复制的是引用值。Java 不会把调用者的变量本身交给方法。

6.6.1 基本类型参数

```
public static void changeNumber(int number) {
    number = 100;
}

int value = 10;
changeNumber(value);
System.out.println(value); // 10
```

方法中的 `number` 是独立副本，修改它不会改变调用者的 `value`。

6.6.2 引用类型参数

```
public static void changeFirst(int[] numbers) {
    numbers[0] = 100;
}

int[] values = {10, 20};
changeFirst(values);
System.out.println(values[0]); // 100
```

形参 `numbers` 得到引用值的副本，但这个副本与 `values` 指向同一个数组对象，因此修改对象内部元素能够被调用者观察到。

6.6.3 重新赋值形参不会改变调用者引用

```
public static void replaceArray(int[] numbers) {
    numbers = new int[]{100, 200};
}

int[] values = {10, 20};
replaceArray(values);
System.out.println(values[0]); // 10
```

方法只是让形参副本改为指向新数组，调用者的 `values` 仍指向原数组。

参数类型	复制的值	方法内操作	调用者能否看到
基本类型	数值本身	给形参重新赋值	不能
引用类型	引用值	修改同一对象的内容	能
引用类型	引用值	让形参指向新对象	不能改变调用者变量的指向

6.7 方法重载

同一个类中，多个方法可以使用相同名称，只要参数列表不同，这称为方法重载。

```
public static int add(int a, int b) {
    return a + b;
}

public static double add(double a, double b) {
    return a + b;
}

public static int add(int a, int b, int c) {
    return a + b + c;
}
```

编译器根据实参数量、类型和顺序选择可调用的方法。

6.7.1 哪些变化可以构成重载

变化	能否构成重载
参数个数不同	可以
参数类型不同	可以
参数类型顺序不同	可以
只有参数名不同	不可以
只有返回值类型不同	不可以
只有访问修饰符不同	不可以

错误示例：

```
int calculate(int value) { return value; }
double calculate(int value) { return value; } // 错误：参数列表相同
```

调用方法时不能只依靠接收变量的类型来区分重载，因此返回值类型不参与重载签名判定。

6.7.2 自动类型转换与歧义

```
static void show(int value) { }
static void show(double value) { }

show(10);   // 选择 show(int)
show(10.5); // 选择 show(double)
```

编译器会优先选择更精确的匹配。如果多个重载都同样合适，调用会产生编译错误。重载数量不宜过多，参数含义也应保持一致。

6.8 完整示例：学生类

```
class Student {
    String name;
    int age;
    double score;

    Student(String name, int age, double score) {
        this.name = name;
        this.age = age;
        this.score = score;
    }

    void introduce() {
        System.out.println("姓名：" + name + "，年龄：" + age);
    }

    boolean isPassed() {
        return score >= 60;
    }
}

public class StudentDemo {
    public static void printResult(Student student) {
        student.introduce();
        System.out.println(student.isPassed() ? "及格" : "不及格");
    }

    public static void main(String[] args) {
        Student first = new Student("张三", 20, 86.5);
        Student second = new Student("李四", 19, 55.0);

        printResult(first);
        printResult(second);
    }
}
```

这个例子中，`Student` 负责保存和处理学生自身数据，`StudentDemo` 负责创建对象并组织程序流程。

常见错误

- 把类和对象混为一谈，认为声明类就已创建对象。
- 创建对象后忘记保存引用，或对 `null` 调用方法。
- 给构造方法写返回值类型，使它变成普通方法。
- 在 `static main` 中直接访问实例字段。
- 把“引用值的副本”误说成 Java 按引用传递。
- 认为给形参重新赋值能改变调用者变量的指向。
- 只改变返回类型或参数名，误以为构成方法重载。

本课小结

- 类描述字段与方法，对象是根据类创建的实例。
- 构造方法在创建对象时初始化状态，`this` 表示当前对象。
- Java 始终采用值传递；引用类型参数复制的是引用值。
- 修改共享对象的内容可以被观察到，重新赋值形参不会改变调用者变量。
- 方法重载要求参数列表不同，返回值类型不参与判定。

自测题

1. 类、对象和对象引用分别是什么？
2. 为什么声明了带参数构造方法后，`new Student()` 可能无法编译？
3. 数组参数为何能修改元素，却不能通过重新赋值形参替换调用者数组？
4. 编写 `print` 的三个重载，分别接收 `int`、`double` 和 `String`。